PROJECT REPORT

FastSpeech2-Based English and Telugu Text-to-Speech Synthesis

Deep Learning | Speech Processing | Neural Speech Generation

Project repository: https://github.com/akankshareddy06/FastSpeech2-English-Telugu-TTS

Report type: Student-friendly technical overview

Scope: English and Telugu speech synthesis implementations

Abstract

Imagine typing a sentence in English or Telugu and hearing it spoken aloud. That is the idea behind this project. We explore how FastSpeech2, a deep-learning model for text-to-speech (TTS), can turn written language into speech. The project is organised into English and Telugu implementations, allowing each language to use its own text processing and model resources. FastSpeech2 predicts an acoustic representation of speech, while a neural vocoder turns that representation into an audio waveform. The repository also contains supporting demonstrations and project materials. Our focus is on explaining the complete speech-generation process, the choices behind the architecture, and the practical challenges of working with two languages. Quantitative claims about naturalness or accuracy are intentionally avoided until measured results have been verified.

Keywords: FastSpeech2, Text-to-Speech, Telugu, English, Deep Learning, Mel-Spectrogram, Neural Vocoder

1. Introduction

Speech is one of the easiest ways for people to interact with technology. TTS systems make written material accessible through audio, helping with reading assistance, educational tools, voice interfaces and digital services. Neural TTS has improved substantially by learning patterns directly from recorded speech rather than relying only on manually crafted rules.

English and Telugu differ in script, pronunciation and linguistic structure. A useful speech-synthesis system must handle these differences before it can generate clear audio. In this project, FastSpeech2 provides the core acoustic modelling approach, and separate English and Telugu workflows make it possible to address language-specific requirements.

2. Problem Statement

Turning text into natural-sounding speech is not simply a matter of reading characters aloud. A model must decide how each sound should be pronounced, how long it should last, and how the pitch and emphasis should change across a sentence. These decisions become even more difficult when working across languages with different writing systems and available datasets. The project therefore explores a FastSpeech2-based workflow that can process English and Telugu text and synthesize corresponding speech.

3. Project Objectives

- Build and organise separate speech-synthesis implementations for English and Telugu.
- Use FastSpeech2 to predict acoustic features from processed text.
- Account for duration, pitch and energy when generating speech.
- Convert predicted mel-spectrograms into playable audio using a compatible vocoder.
- Review generated examples and training artefacts to understand model behaviour.
- Document practical limitations and identify ways to improve quality and usability.

4. Background and Approach

Older TTS approaches often combined recorded speech fragments or used statistical models to generate speech parameters. Later neural systems improved output quality, but some autoregressive models generate acoustic frames sequentially, which can slow down inference. FastSpeech2 uses a non-autoregressive approach, enabling parallel prediction of mel-spectrogram frames. Its variance adaptor explicitly models speech duration, pitch and energy, giving the system useful signals for rhythm and expression.

5. System Architecture

Stage	What happens
1. Text input	User provides an English or Telugu sentence.
2. Preprocessing	Text is normalized and converted into language-appropriate tokens or phonemes.
3. FastSpeech2 encoder	The input sequence is transformed into contextual hidden representations.
4. Variance adaptor	Duration, pitch and energy information guides the acoustic sequence.
5. Decoder	A mel-spectrogram is predicted.
6. Neural vocoder	The mel-spectrogram is converted to a waveform and saved or played.

6. Technologies and Project Components

- **Python and PyTorch:** support model implementation, data preparation, training and inference.
- **FastSpeech2:** predicts the acoustic representation used for synthesizing speech.
- **Mel-spectrograms:** represent the changing frequency characteristics of speech over time.
- **Vocoder assets:** the project includes HiFi-GAN-related resources for waveform synthesis.

- **Audio and visual outputs:** WAV samples and plots help with listening checks and model analysis.

7. Datasets and Language Processing

The project materials identify Svarah for the English workflow and IndicTTS Telugu for the Telugu workflow. The two languages require careful handling of written symbols, pronunciation, and audio-text alignment. A typical preparation pipeline involves checking transcripts, normalizing audio, building token or phoneme sequences, extracting mel-spectrograms, and preparing the timing and acoustic features needed for training. The final report should use the actual configuration files to state exact speaker counts, recording hours, training splits and sampling rates.

8. Implementation Workflow

- **Prepare the data.** Organize speech recordings and matching transcripts for each language.
- **Extract features.** Generate acoustic features and training targets for duration, pitch and energy as required by the implementation.
- **Configure the model.** Set language-specific preprocessing and FastSpeech2 parameters.
- **Train or load weights.** Use trained checkpoints to predict acoustic representations from unseen text.
- **Synthesize audio.** Pass predicted mel-spectrograms through the vocoder.
- **Inspect outputs.** Listen to generated WAV files and review loss and acoustic plots.

9. Results and Discussion

The repository overview describes audio demonstrations and visualisations for the English and Telugu workflows, including training/validation losses and duration, pitch and energy plots. These are useful for showing that the system has an end-to-end workflow and for discussing how its acoustic predictions behave. However, the presence of sample audio alone does not establish that the speech is natural or intelligible for all inputs.

A stronger evaluation would include a structured listening test, pronunciation checks for Telugu and English, synthesis-speed measurements and comparisons using the same test sentences. No numerical Mean Opinion Score (MOS), accuracy percentage, inference speed or objective intelligibility result is claimed here because those measurements have not been verified.

10. Applications

A system like this could support accessible reading tools, educational narration, spoken notifications, language-learning applications and voice-enabled interfaces. Telugu speech synthesis is particularly relevant to digital experiences where users prefer to hear information in their native language.

11. Strengths and Limitations

Strengths. FastSpeech2 separates acoustic prediction from waveform generation, permits parallel generation of acoustic frames and includes explicit control signals for duration, pitch and energy. Maintaining separate implementations also helps the project account for language-specific preprocessing.

Limitations. Speech quality depends heavily on the amount and cleanliness of training data. Pronunciation errors can come from text processing, while unnatural rhythm or voice quality can come from acoustic prediction or vocoder mismatch. Large checkpoint files also complicate distribution; the GitHub upload discussed for this project excluded the two approximately 400 MB language checkpoints, so code availability should not be

confused with a complete downloadable trained-model package.

12. Future Improvements

- Conduct human listening tests with native English and Telugu speakers.
- Improve text normalization and pronunciation handling for Telugu words and names.
- Compare FastSpeech2 output with other current TTS approaches using matched evaluation settings.
- Measure inference time, memory use and robustness to longer sentences.
- Add a simple browser interface and clear setup instructions.
- Publish reproducible model-download guidance and dataset/license information.

13. Conclusion

This project shows how a modern neural speech-synthesis pipeline can be organised for English and Telugu. FastSpeech2 takes processed language input and predicts an acoustic representation, while a vocoder produces the final audio. The most valuable part of the work is the connection between language processing, acoustic modelling and practical audio output. The next step is to verify objective and human-listening results so that claims about speed, clarity and naturalness are supported by repeatable experiments.

14. References

1. Ren, Y. et al. (2020). *FastSpeech 2: Fast and High-Quality End-to-End Text to Speech.* arXiv:2006.04558.

2. Kong, J., Kim, J. and Bae, J. (2020). *HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis.* NeurIPS.

3. Vaswani, A. et al. (2017). *Attention Is All You Need.* NeurIPS.

4. Project repository: https://github.com/akankshareddy06/FastSpeech2-English-Telugu-TTS

Verification note: Architecture explanations reflect standard FastSpeech2 practice. Exact dataset statistics, experiment settings and numerical evaluation should be checked against the project files before formal academic submission.